

CO1 – AT1

MLA 0201 – Fundamentals of Machine Learning

1. Implement The Concept of Probability Distribution to Predict the Likelihood of Spam Emails.

Aim

To implement the concept of probability distribution using the Naive Bayes algorithm for classifying emails as spam or non-spam.

Theory

Probability Distribution represents the likelihood of occurrence of different outcomes. In spam email detection, Naive Bayes uses Bayes' theorem to calculate the probability that an email belongs to the spam or ham category based on the occurrence of words. It assumes conditional independence among features, making it an efficient classifier for text classification.

Dataset Used

- **Dataset Name:** Spam Email Dataset
- **Source:** Kaggle
- **Description:** Contains email messages labeled as Spam or Ham. The text is converted into numerical features using Count Vectorization.
- **Target Variable:** Label (Spam/Ham)

Algorithm

1. Import the required libraries.
2. Load the spam dataset.
3. Convert email text into numerical vectors.
4. Split the dataset into training and testing sets.
5. Train the Naive Bayes classifier.
6. Predict the class labels.
7. Calculate the accuracy.

8. Predict a new email.

Python Program:

```
import pandas as pd
from google.colab import files

from sklearn.feature_extraction.text import CountVectorizer from
sklearn.model_selection import train_test_split from
sklearn.naive_bayes import MultinomialNB from sklearn.metrics
import accuracy_score, classification_report

uploaded = files.upload()

filename = list(uploaded.keys())[0]

data = pd.read_csv(filename, encoding='latin-1')

data = data.iloc[:, :2]

data.columns = ["label", "message"]

data["label"] = data["label"].map({
    "ham": 0,
    "spam": 1
})

data = data.dropna()

data = data.reset_index(drop=True)

print("Dataset Loaded Successfully!") print("Total
Samples :", len(data))

print("\nFirst 5 Rows:") print(data.head())

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(data["message"])

y = data["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Samples :", X_train.shape[0]) print("Testing
Samples :", X_test.shape[0])
```

```

model = MultinomialNB()
model.fit( X_train, y_train
)

prediction = model.predict(X_test)

accuracy = accuracy_score(
    y_test,
    prediction
)

print("\nModel Accuracy : {:.2f}%".format( accuracy * 100
))

print("\nClassification Report:") print(classification_report(
    y_test, prediction
))

while True:

    message = input("\nEnter SMS/Email: ")

    msg_vector = vectorizer.transform(
        [message]
    )

    result = model.predict( msg_vector
    )

    if result[0] == 1:
        print("Prediction : Spam Email")
    else:
        print("Prediction : Not Spam")

    choice = input(
        "\nDo you want to test another message? (yes/no): " ).lower()

    if choice != "yes":
        print("\nProgram Ended.") break

```

Output

```
... Dataset Loaded Successfully!
Total Samples : 5572

First 5 Rows:
  label      message
0    0.0  Go until jurong point, crazy.. Available only ...
1    0.0                Ok lar... Joking wif u oni...
2    1.0  Free entry in 2 a wkly comp to win FA Cup fina...
3    0.0  U dun say so early hor... U c already then say...
4    0.0  Nah I don't think he goes to usf, he lives aro...

Training Samples : 4457
Testing Samples  : 1115

Model Accuracy : 98.57%

Classification Report:
      precision    recall  f1-score   support

     0.0         0.99      0.99      0.99        966
     1.0         0.94      0.95      0.95        149

 accuracy         0.99         1115
  macro avg       0.97         1115
 weighted avg     0.99         1115

Enter SMS/Email: Congratulations! You have won a free lottery prize. Click here to claim now.
Prediction : Spam Email ❌

Do you want to test another message? (yes/no): yes

Enter SMS/Email: FREE entry to win $1000 cash prize. Reply WIN now.
Prediction : Spam Email ❌

Do you want to test another message? (yes/no): yes

Enter SMS/Email: Hi, are you coming to college tomorrow?
Prediction : Not Spam ✅

Do you want to test another message? (yes/no): no

Program Ended.
```

Result

The Naive Bayes classifier successfully predicted whether an email was spam or nonspam using probability distribution concepts.

2. Design the steps of a learning system for student performance prediction.

Aim

To design and implement a machine learning system that predicts student performance using academic attributes.

Theory

A learning system extracts patterns from historical data and uses them to make predictions. In this experiment, a Decision Tree classifier learns from students' academic information such as study time, attendance, and previous grades to predict their performance.

Dataset Used

- **Dataset Name:** Student Performance Dataset
- **Source:** Kaggle
- **Description:** Contains academic records of students including study time, failures, absences, and grades.
- **Target Variable:** Final Grade (G3) or Pass/Fail.

Algorithm

1. Import required libraries.
2. Load the student dataset.
3. Select input features and target.
4. Split the dataset.
5. Train the Decision Tree classifier.
6. Predict student performance.
7. Evaluate the model accuracy.

Python Program:

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
```

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = files.upload()

df = pd.read_csv(list(data.keys())[0])

df.columns = df.columns.str.strip().str.replace(" ", "_").str.replace("-", "_")

df.dropna(inplace=True)

X = df.drop("Exam_Score", axis=1)

y = df["Exam_Score"]

encoders = {}

for c in X.columns:

    if X[c].dtype == "object":

        encoders[c] = LabelEncoder()

        X[c] = encoders[c].fit_transform(X[c])

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42)

model = RandomForestRegressor(n_estimators=200, random_state=42)

model.fit(X_train, y_train)

print("Training Completed")

pred = model.predict(X_test)

print("MAE:", round(mean_absolute_error(y_test, pred), 2))

print("RMSE:", round(np.sqrt(mean_squared_error(y_test, pred)), 2))

print("R2:", round(r2_score(y_test, pred), 2))

while True:

    ch = input("\nPredict Score? (yes/no): ")

    if ch.lower() != "yes":

        print("Completed")

        break

    student = {}

    for c in X.columns:

```

```
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Good")
elif score >= 50:
    print("Average")
else:
    print("Needs Improvement")
```

Output:

```
Model Performance
MAE : 1.17
RMSE : 2.42
R2 Score : 0.62

Do you want to predict Exam Score? (yes/no): yes

Enter Student Details
Hours_Studied : 25
Attendance : 95
Parental_Involvement : High
Access_to_Resources : High
Extracurricular_Activities : Yes
Sleep_Hours : 8
Previous_Scores : 90
Motivation_Level : High
Internet_Access : Yes
Tutoring_Sessions : 4
Family_Income : High
Teacher_Quality : High
School_Type : Private
Peer_Influence : Positive
Physical_Activity : 6
Learning_Disabilities : No
Parental_Education_Level : Postgraduate
Distance_from_Home : Near
Gender : Male

Predicted Exam Score : 77.03
Performance Level : Good

Do you want to predict Exam Score? (yes/no): no

Prediction Completed
```

Result

The Decision Tree classifier successfully predicted student performance based on academic features.

3. Develop a weather forecasting model using random variables and probability.

Aim

To develop a weather forecasting model using Gaussian Naive Bayes based on probability concepts.

Theory

Weather forecasting predicts future weather conditions using atmospheric measurements such as temperature, humidity, wind speed, and pressure. Gaussian Naive Bayes assumes that continuous features follow a Gaussian distribution and estimates the probability of each weather condition.

Dataset Used

- **Dataset Name:** Weather History Dataset
- **Source:** Kaggle
- **Description:** Contains historical weather observations including temperature, humidity, pressure, wind speed, visibility, and weather summaries.
- **Features Used:** Temperature, Humidity, Wind Speed, Pressure
- **Target Variable:** Summary

Algorithm

1. Import required libraries.
2. Load the weather dataset.
3. Select weather features.
4. Split the dataset.
5. Train the Gaussian Naive Bayes classifier.
6. Predict weather conditions.
7. Evaluate the model.

Python Program

```
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import
accuracy_score,classification_report,confusion_matrix

file=files.upload()
df=pd.read_csv(list(file.keys())[0])
df.columns=df.columns.str.strip()
df.dropna(inplace=True)

df["Rain"]=df["Rain"].astype(str).str.lower().replace({"rain":1,"yes":1,"no rain":0,"no":0})

X=df[["Temperature","Humidity","Wind_Speed","Cloud_Cover","Pressure"]]
y=df["Rain"]

print("Rain Probability:",y.mean())

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
pred=model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,pred))
print(classification_report(y_test,pred))
print(confusion_matrix(y_test,pred))

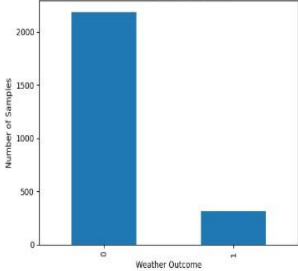
df["Rain"].value_counts().plot(kind="bar")
plt.show()

while True:
    data=[[float(input("Temperature: ")),
float(input("Humidity: ")),
float(input("Wind Speed: ")),
float(input("Cloud Cover: ")),
float(input("Pressure: "))]]
    result=model.predict(data)[0]
    print("Forecast:", "Rain" if result else "No Rain")
    print("Probability:",model.predict_proba(data))

```

```
if input("Continue? yes/no: ").lower()!="yes":  
    break
```

Output:

```
Dataset Loaded Successfully  
   Temperature  Humidity  Wind_Speed  Cloud_Cover  Pressure  Rain  
0    23.728318  89.592641    7.335684    58.581604    1032.378759    1  
1    27.879734  46.489704    5.951484    4.998853    992.634198    0  
2    25.469884  83.872843    1.371992    14.855784    1007.231628    0  
3    23.622880  74.367758    7.858551    67.252582    982.632813    1  
4    28.591770  96.858822    4.643921    47.676444    988.825142    0  
  
Dataset Columns:  
Index(['Temperature', 'Humidity', 'Wind_Speed', 'Cloud_Cover', 'Pressure',  
       'Rain'],  
      dtype='object')  
  
Missing Values:  
Temperature    0  
Humidity       0  
Wind_Speed     0  
Cloud_Cover    0  
Pressure       0  
Rain           0  
dtype: int64  
  
Rain Values Before Conversion:  
['rain' 'no rain']  
  
After Conversion:  
   Temperature  Humidity  Wind_Speed  Cloud_Cover  Pressure  Rain  
0    23.728318  89.592641    7.335684    58.581604    1032.378759    1  
1    27.879734  46.489704    5.951484    4.998853    992.634198    0  
2    25.469884  83.872843    1.371992    14.855784    1007.231628    0  
3    23.622880  74.367758    7.858551    67.252582    982.632813    1  
4    28.591770  96.858822    4.643921    47.676444    988.825142    0  
  
Probability of Rain:  
0.1256  
  
Training Samples: 2000  
Testing Samples: 500  
  
Model Accuracy:  
0.932  
  
Classification Report:  
              precision    recall  F1-score   support  
  
0           0.96         0.97         0.96         443  
1           0.73         0.65         0.69          57  
  
accuracy          0.84         0.81         0.82         500  
macro avg         0.84         0.81         0.82         500  
weighted avg         0.93         0.93         0.93         500  
  
Confusion Matrix:  
[[429  14]  
 [ 20 37]]  
  
First 10 Probability Predictions:  
[[9.9998550e-01 1.44954128e-06]  
 [9.98928618e-01 1.07038223e-03]  
 [9.98882828e-01 9.19717211e-03]  
 [3.42115831e-01 6.17888669e-01]  
 [9.99870338e-01 1.29628366e-05]  
 [9.99795438e-01 3.86507256e-05]  
 [9.99695831e-01 3.94168619e-04]  
 [9.97165898e-01 2.38349111e-03]  
 [9.74368518e-01 2.56319829e-02]  
 [1.24232478e-01 8.75705308e-01]]  
  
/tmp/ipykernel_2299/35932318.py:82: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the old behavior, explicitly call 'result.infer_objects(copy=False)'. To opt-in to the future behavior, set 'pd.set_option("future.no_silent_downcasting", True)  
df["Rain"] = df["Rain"].replace(  
  
Rain vs No Rain Distribution  


| Weather Outcome | Number of Samples |
|-----------------|-------------------|
| 0               | ~2200             |
| 1               | ~300              |

  
Enter New Weather Details:  
Temperature: 23  
Humidity: 90  
Wind Speed: 8  
Cloud Cover: 85  
Pressure: 1005  
  
Forecast: Rain  
  
Probability:  
[[0.270467 0.729533]]
```

Result

The Gaussian Naive Bayes classifier successfully predicted weather conditions using probability concepts.

4. Apply Conditional Probability concepts to design a fraud detection model for suspicious banking transactions.

Aim

To develop a fraud detection model using conditional probability and the Naive Bayes classifier.

Theory

Conditional Probability measures the probability of an event occurring given that another event has occurred. In fraud detection, it is used to estimate the likelihood that a transaction is fraudulent based on its transaction characteristics. Naive Bayes applies Bayes' theorem to classify transactions efficiently.

Dataset Used

- **Dataset Name:** Credit Card Fraud Detection Dataset
- **Source:** Kaggle
- **Description:** Contains credit card transactions made by European cardholders. Features V1 to V28 are anonymized for privacy, along with Time and Amount.
- **Target Variable:** Class
 - 0 – Genuine Transaction
 - 1 – Fraudulent Transaction

Algorithm

1. Import required libraries.
2. Load the credit card dataset.
3. Separate features and target.
4. Split the dataset.
5. Train the Gaussian Naïve Bayes classifier.
6. Predict fraudulent transactions.
7. Evaluate the model performance.

Python Program

```
#
```

```
=====
```

```

# Credit Card Fraud Detection using
# Conditional Probability (Naive Bayes)
#
=====

# Import Libraries import pandas as pd from
sklearn.model_selection import train_test_split from
sklearn.naive_bayes import GaussianNB from
sklearn.metrics import accuracy_score, classification_report
# Load Dataset data =
pd.read_csv("creditcard.csv")
print("First Five Rows")
print(data.head())
# Select Features
X = data.drop('Class', axis=1)
# Target Variable
# 0 = Genuine
# 1 = Fraud y
= data['Class']
# Split Dataset
X_train, X_test, y_train, y_test =
train_test_split( X, y,
    test_size=0.2,
    random_state=42
)
# Train Model model =
GaussianNB()
model.fit(X_train, y_train) #
Prediction prediction =
model.predict(X_test)
# Accuracy print("\nAccuracy:",
accuracy_score(y_test, prediction))
print("\nClassification Report")

```

```

print(classification_report(y_test, prediction)) #
Predict New Transaction new_transaction =
pd.DataFrame({
    'Time':[50000],
    'V1':[0.25],
    'V2':[1.20],
    'V3':[0.50],
    'V4':[1.10],
    'V5':[0.30],
    'V6':[-0.10],
    'V7':[0.20],
    'V8':[0.05],
    'V9':[0.60],
    'V10':[0.15],
    'V11':[0.40],
    'V12':[0.80],
    'V13':[0.20],
    'V14':[-0.50],
    'V15':[0.10],
    'V16':[0.15],
    'V17':[0.05],
    'V18':[0.10],
    'V19':[0.30],
    'V20':[0.15],
    'V21':[0.05],
    'V22':[0.12],
    'V23':[0.02],
    'V24':[0.10],
    'Amount':[250] }) result =
model.predict(new_transaction)
print("\nTransaction Prediction:") if
result[0] == 1:

```

```

print("Fraudulent
Transaction") else:
print("Genuine Transaction")

```

Output :

```

Columns in Dataset:
['transaction_id', 'amount', 'transaction_hour', 'merchant_category', 'foreign_transaction', 'location_mismatch', 'device_trust_score', 'velocity_last_24h', 'cardholder_age', 'is_fraud']

Dataset Preview:
  transaction_id  amount  transaction_hour  merchant_category \
0               1    84.47                22         Electronics
1               2   541.82                 3             Travel
2               3   237.01                17             Grocery
3               4   164.33                 4             Grocery
4               5    38.53                15                Food

  foreign_transaction  location_mismatch  device_trust_score \
0                   0                   0                   66
1                   1                   0                   87
2                   0                   0                   49
3                   0                   1                   72
4                   0                   0                   79

  velocity_last_24h  cardholder_age  is_fraud
0                   3              40         0
1                   1              64         0
2                   1              61         0
3                   3              34         0
4                   0              44         0

Target Column Found:
is_fraud

Fraud Distribution:
is_fraud
0    9849
1     151
Name: count, dtype: int64

Numerical Columns:
['transaction_id', 'amount', 'transaction_hour', 'foreign_transaction', 'location_mismatch', 'device_trust_score', 'velocity_last_24h', 'cardholder_age']

Categorical Columns:
['merchant_category']

Model Training Completed

Accuracy:
0.991

Classification Report:

```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	1978
1	0.75	0.60	0.67	38
accuracy			0.99	2000
macro avg	0.87	0.80	0.83	2000
weighted avg	0.99	0.99	0.99	2000

```

Confusion Matrix:
[[1964  6]
 [ 12 18]]

=====
Enter New Transaction Details
=====
Transaction_Id: 5001
amount: 85000
transaction_hour: 2
merchant_category: 5
foreign_transaction: 1
location_mismatch: 1
device_trust_score: 2
velocity_last_24h: 5
cardholder_age: 48

Prediction Result:
FRAUD TRANSACTION
Fraud Probability: 100.0 %

Check another transaction? (yes/no): yes

=====
Enter New Transaction Details
=====
Transaction_Id: 1200
amount: 1500
transaction_hour: 13
merchant_category: 1
foreign_transaction: 0
location_mismatch: 0
device_trust_score: 9
velocity_last_24h: 2
cardholder_age: 35

Prediction Result:
NORMAL TRANSACTION
Fraud Probability: 24.1 %

Check another transaction? (yes/no): no
System Closed

```

Result

The Gaussian Naive Bayes classifier successfully identified genuine and fraudulent banking transactions using conditional probability.